

"""FastAPI 应用入口（v3 内控评价智能审核系统）。

清理过程中：仅暴露 auth/users/audit-logs；v3 新路由将在 commit 2/3 引入。

"""

from __future__ import annotations

from pathlib import Path

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from fastapi.responses import FileResponse

from fastapi.staticfiles import StaticFiles

from app.api.audit_routes import findings_router, materials_router, tasks_router, units_router

from app.api.auth_routes import audit_router, auth_router, users_router

from app.api.knowledge_routes import checkitems_router, indicators_router

from app.api.regulation_routes import regulations_router

from app.api.settings_routes import settings_router

from app.core.config import settings

from app.models import init_db

app = FastAPI(title=settings.app_name, version="3.0.0")

app.add_middleware(

CORSMiddleware,

allow_origins=["*"],

allow_methods=["*"],

allow_headers=["*"],

)

app.include_router(auth_router)

app.include_router(users_router)

app.include_router(audit_router)

app.include_router(settings_router)

app.include_router(indicators_router)

app.include_router(checkitems_router)

app.include_router(regulations_router)

app.include_router(units_router)

app.include_router(tasks_router)

app.include_router(findings_router)

app.include_router(materials_router)

@app.on_event("startup")

def _startup() -> None:

init_db()

静态前端

_FRONTEND_DIR = Path(__file__).resolve().parents[2] / "frontend"

```
if not _FRONTEND_DIR.exists():
    _FRONTEND_DIR = Path("/frontend")

if _FRONTEND_DIR.exists():
    app.mount("/static", StaticFiles(directory=_FRONTEND_DIR), name="static")

    @app.get("/")
    def index():
        index_html = _FRONTEND_DIR / "index.html"
        if index_html.exists():
            return FileResponse(index_html)
        return {"name": settings.app_name, "docs": "/docs"}

    @app.get("/favicon.ico", include_in_schema=False)
    def favicon():
        svg = _FRONTEND_DIR / "favicon.svg"
        if svg.exists():
            return FileResponse(svg, media_type="image/svg+xml")
        return FileResponse(svg, status_code=404)
else:
    @app.get("/")
    def root() -> dict:
        return {"name": settings.app_name, "docs": "/docs"}

@app.get("/api/health")
def health() -> dict:
    return {
        "status": "ok",
        "app": settings.app_name,
        "llm_default_provider": settings.llm_provider,
        "embedder": settings.embedder,
        "vector_store": settings.vector_store,
    }

"""全局配置（v3 内控评价智能审核系统）。

所有外部依赖通过环境变量切换；默认走「离线回退实现」，零外部依赖可运行。
"""
from __future__ import annotations

from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

BACKEND_DIR = Path(__file__).resolve().parents[2]

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
```

```
app_name: str = "内控评价智能审核系统"
debug: bool = True

# 数据库
database_url: str = f"sqlite:///{{BACKEND_DIR / 'audit.db'}}"

# 对象存储
storage_dir: str = str(BACKEND_DIR / "storage")

# Embedding: bge | stub
embedder: str = "stub"
embedding_dim: int = 256
bge_model_name: str = "BAAI/bge-large-zh-v1.5"

# 向量库: memory | qdrant
vector_store: str = "memory"
qdrant_url: str = "http://localhost:6333"

# ===== LLM 配置 =====
# provider: stub | deepseek | claude
# 这些值是「默认值」, 运行时管理员可通过 /api/settings/llm 在数据库覆盖
llm_provider: str = "deepseek"
llm_model: str = "deepseek-v4-pro"
llm_base_url: str = "https://api.deepseek.com/v1"
llm_api_key: str = ""
llm_thinking_mode: str = "non_think" # non_think | think_high | think_max
# Claude 兼容
anthropic_api_key: str = ""

# Celery / Redis
redis_url: str = "redis://localhost:6379/0"
celery_eager: bool = True

@lru_cache
def get_settings() -> Settings:
    s = Settings()
    Path(s.storage_dir).mkdir(parents=True, exist_ok=True)
    return s

settings = get_settings()
"""v3 领域常量。

5 大检查维度 (来自 v3 §1.3) + 风险等级 + 状态枚举。
"""
from __future__ import annotations

import enum
```

```
class Severity(str, enum.Enum):
    HIGH = "高"
    MEDIUM = "中"
    LOW = "低"

# 维度 (v3 §1.3)
DIMENSIONS = [
    "总体合规性",      # 真实性/年度一致性/正式性/要素完整性
    "相关性核查",      # 6 大业务分类
    "评分合规性",      # 轮岗扣分、不适用业务、整改时点、补充指标=10
    "复核规范性",      # 复核权限/分离/内容/流程
    "报告编报合规性",  # 报告信息/内控描述/整改填报/系统操作
]

# Finding 的 finding_type (细分类, 便于报告聚合)
FINDING_TYPES = [
    "真实性问题",
    "年度一致性问题",
    "正式性问题",
    "完整性问题",
    "相关性问题",
    "合规性问题",
    "评分合规问题",
    "复核规范问题",
    "报告编报问题",
]

# 6 大业务分类 (v3 §1.3 维度二)
BUSINESS_CATEGORIES = [
    "组织层面",      # 三重一大、轮岗、内控会议、内控培训
    "预算业务",      # 预算制度、公开、绩效管理
    "收支业务",      # 票据台账、支出审批
    "政府采购",      # 采购全流程
    "资产建设合同",  # 盘点表、项目全流程、合同台账
    "内部监督",      # 检查方案、整改闭环
]

# 任务状态机
class TaskStatus(str, enum.Enum):
    PENDING = "pending"      # 任务已创建, 材料还未上传完成
    RUNNING = "running"      # AI 核查中
    AI_DONE = "ai_done"      # AI 初核完成, 待人工复核
    REVIEWING = "reviewing"  # 人工复核中
    FINALIZED = "finalized"  # 已定稿, 已生成正式报告
    ARCHIVED = "archived"    # 已归档 (整改闭环)
    FAILED = "failed"
```

```
# Finding 复核状态
class ReviewStatus(str, enum.Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    IGNORED = "ignored"
    ADJUSTED = "adjusted"

# 整改状态
class RectificationStatus(str, enum.Enum):
    OPEN = "open"
    SUBMITTED = "submitted"
    RESOLVED = "resolved"
"""v3 角色与权限 (v3 §3.7) 。

四角色:
- super_admin    超级管理员: 全部权限 (管理知识库、用户、任务)
- auditor        审查员: 创建任务、查看结果、复核标注、生成报告
- unit           被检查单位: 上传材料、查看本单位核查结果、填写整改说明
- readonly       只读用户: 查看报告, 不可操作
"""
from __future__ import annotations

ROLE_SUPER_ADMIN = "super_admin"
ROLE_AUDITOR = "auditor"
ROLE_UNIT = "unit"
ROLE_READONLY = "readonly"

# 向后兼容: 旧种子 admin 角色直接映射到 super_admin
_LEGACY_ROLE_MAP = {"admin": ROLE_SUPER_ADMIN}

ALL_ROLES = frozenset([ROLE_SUPER_ADMIN, ROLE_AUDITOR, ROLE_UNIT, ROLE_READONLY])

def normalize_role(role: str) -> str:
    return _LEGACY_ROLE_MAP.get(role, role)

def role_label(role: str) -> str:
    return {
        ROLE_SUPER_ADMIN: "超级管理员",
        ROLE_AUDITOR: "审查员",
        ROLE_UNIT: "被检查单位",
        ROLE_READONLY: "只读用户",
    }.get(normalize_role(role), role)

def is_admin(role: str) -> bool:
```

```
    return normalize_role(role) == ROLE_SUPER_ADMIN

def is_auditor_or_above(role: str) -> bool:
    r = normalize_role(role)
    return r in (ROLE_SUPER_ADMIN, ROLE_AUDITOR)

def is_unit(role: str) -> bool:
    return normalize_role(role) == ROLE_UNIT

def can_manage_knowledge(role: str) -> bool:
    """管理知识库（指标库、问题清单库）：仅超级管理员。"""
    return is_admin(role)

def can_create_task(role: str) -> bool:
    """创建核查任务：审查员及以上。"""
    return is_auditor_or_above(role)

def can_review_findings(role: str) -> bool:
    """复核标注 finding：审查员及以上。"""
    return is_auditor_or_above(role)

def can_rectify_findings(role: str) -> bool:
    """填写整改说明：被检查单位 + 审查员（代填）。"""
    r = normalize_role(role)
    return r in (ROLE_SUPER_ADMIN, ROLE_AUDITOR, ROLE_UNIT)
"""认证与审计：从 Authorization Bearer token 解析当前用户。

简单 token：DB 存随机字符串，请求头携带，无需 JWT 库。
"""
from __future__ import annotations

import secrets
from typing import Optional

from fastapi import Depends, Header, HTTPException, status
from sqlalchemy.orm import Session

from app.core.permissions import is_admin, is_auditor_or_above
from app.models import AuditLog, AuthToken, User, get_db

def new_token() -> str:
    return secrets.token_urlsafe(32)
```

```
def _extract_token(authorization: Optional[str]) -> Optional[str]:
    if not authorization:
        return None
    parts = authorization.split()
    if len(parts) == 2 and parts[0].lower() == "bearer":
        return parts[1]
    return None

def get_current_user(
    authorization: Optional[str] = Header(default=None),
    db: Session = Depends(get_db),
) -> User:
    token = _extract_token(authorization)
    if not token:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="未登录",
            headers={"WWW-Authenticate": "Bearer"},
        )
    record = db.query(AuthToken).filter(AuthToken.token == token).first()
    if record is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "登录已失效")
    user = db.get(User, record.user_id)
    if user is None or not user.is_active:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "账号已停用")
    return user

def require_admin(user: User = Depends(get_current_user)) -> User:
    if not is_admin(user.role):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "需要超级管理员权限")
    return user

def require_auditor(user: User = Depends(get_current_user)) -> User:
    """审查员及以上（含超级管理员）。"""
    if not is_auditor_or_above(user.role):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "需要审查员权限")
    return user

def log_action(db: Session, user: Optional[User], action: str,
               target_type: str = "", target_id: Optional[int] = None,
               detail: str = "") -> None:
    """记录审计日志（§3.7「全程留痕可溯源」）。"""
    db.add(AuditLog(
        user_id=user.id if user else None,
        username=user.username if user else "",
```

```
        action=action,
        target_type=target_type,
        target_id=target_id,
        detail=detail[:1000],
    ))
    # 不在此 commit, 由调用方与业务一并提交
    """数据库引擎与会话。SQLite (本地) / PostgreSQL (生产) 通过 DATABASE_URL 切换。"""
    from __future__ import annotations

    from typing import Iterator

    from sqlalchemy import create_engine
    from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

    from app.core.config import settings

    _connect_args = {"check_same_thread": False} if settings.database_url.startswith("sqlite") else {}

    engine = create_engine(settings.database_url, connect_args=_connect_args, future=True)
    SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False, future=True)

    class Base(DeclarativeBase):
        pass

    def init_db() -> None:
        """建表 (首次初始化) + 老库自动补列 + 种子管理员账号。"""
        import app.models.entities # noqa: F401 确保模型注册

        Base.metadata.create_all(bind=engine)
        _ensure_columns()
        _seed_admin()

    # 历次新增的列 (旧库自动补全)
    _EXTRA_COLUMNS: list[tuple[str, str, str]] = [
        # (table, column, ddl)
        ("indicators", "audit_points", "TEXT NOT NULL DEFAULT ''"),
        ("materials", "content_hash", "VARCHAR(64) NOT NULL DEFAULT ''"),
        ("materials", "content_fingerprint", "VARCHAR(64) NOT NULL DEFAULT ''"),
        ("audit_tasks", "progress_current", "INTEGER NOT NULL DEFAULT 0"),
        ("audit_tasks", "progress_total", "INTEGER NOT NULL DEFAULT 0"),
        ("audit_tasks", "progress_text", "VARCHAR(256) NOT NULL DEFAULT ''"),
        ("audit_tasks", "fast_mode", "BOOLEAN NOT NULL DEFAULT FALSE"),
        ("worksheet_rows", "adjustment_note", "TEXT NOT NULL DEFAULT ''"),
    ]

    def _ensure_columns() -> None:
```



```
"""对存量库等地 ALTER TABLE ADD COLUMN (SQLite/PG 兼容子集) 。"""
from sqlalchemy import inspect, text
insp = inspect(engine)
existing_tables = set(insp.get_table_names())
for table, col, ddl in _EXTRA_COLUMNS:
    if table not in existing_tables:
        continue
    cols = {c["name"] for c in insp.get_columns(table)}
    if col in cols:
        continue
    with engine.begin() as conn:
        conn.execute(text(f"ALTER TABLE {table} ADD COLUMN {col} {ddl}"))

def _seed_admin() -> None:
    """首次运行时创建默认管理员账号 (admin / admin123) 。

    生产部署后应立即在前端修改密码 (Phase 4 余下功能) 。
    """
    from app.core.security import hash_password
    from app.models.entities import User

    db = SessionLocal()
    try:
        if db.query(User).first() is not None:
            return
        db.add(User(
            username="admin",
            password_hash=hash_password("admin123"),
            role="super_admin",
            full_name="系统管理员",
            is_active=True,
        ))
        db.commit()
    finally:
        db.close()

def get_db() -> Iterator[Session]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

"""业务实体 (v3: 内控评价智能审核系统) 。

v3 角色: 超级管理员 / 审查员 / 被检查单位 / 只读用户。
v3 核心: 评价指标库 + 问题清单库 + 核查任务 + 核查发现 + 整改闭环。
"""
from __future__ import annotations
```

```
from datetime import datetime
from typing import List, Optional

from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base

# =====
# 用户 & 认证 & 审计 & 系统设置
# =====
class User(Base):
    """用户 + 角色 (v3 §3.7) : super_admin | auditor | unit | readonly. """
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    username: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    role: Mapped[str] = mapped_column(String(32))
    full_name: Mapped[str] = mapped_column(String(64), default="")
    # 若为 unit 角色, 绑定对应被检查单位 (仅能看自己单位的数据)
    unit_id: Mapped[Optional[int]] = mapped_column(
        ForeignKey("audit_units.id"), nullable=True
    )
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

class AuthToken(Base):
    __tablename__ = "auth_tokens"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    token: Mapped[str] = mapped_column(String(128), unique=True, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

class AuditLog(Base):
    """操作审计日志 (v3 §3.7「全程留痕可溯源」) 。"""
    __tablename__ = "audit_logs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[Optional[int]] = mapped_column(Integer, nullable=True, index=True)
    username: Mapped[str] = mapped_column(String(64), default="")
    action: Mapped[str] = mapped_column(String(64))
    target_type: Mapped[str] = mapped_column(String(32), default="")
    target_id: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    detail: Mapped[str] = mapped_column(Text, default="")
```

```
        created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

class AppSetting(Base):
    """系统全局配置 key/value 存储（如 LLM API Key）。
    敏感配置如 API Key 也存在于这里（数据库级隔离即可，避免环境变量层级硬编码）。
    """
    __tablename__ = "app_settings"

    key: Mapped[str] = mapped_column(String(64), primary_key=True)
    value: Mapped[str] = mapped_column(Text, default="")
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), onupdate=func.now()
    )

# =====
# 评价指标库 & 问题清单库（v3 §3.1、§3.2 黄金数据）
# =====
class Indicator(Base):
    """评价指标（编报指南 附件1/附件2）。
    例：1-1-1「三重一大」决策制度建立与执行情况，4 分。
    """
    __tablename__ = "indicators"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    indicator_code: Mapped[str] = mapped_column(String(32), unique=True, index=True)
    # 层级分类（组织层面/业务层面/内部监督；单位评价/部门评价）
    level: Mapped[str] = mapped_column(String(32), default="") # 单位 | 部门
    category: Mapped[str] = mapped_column(String(64), default="") # 6 大业务分类
    subcategory: Mapped[str] = mapped_column(String(64), default="")
    name: Mapped[str] = mapped_column(String(256))
    description: Mapped[str] = mapped_column(Text, default="")
    max_score: Mapped[float] = mapped_column(default=0.0)
    audit_points: Mapped[str] = mapped_column(Text, default="") # 核查要点（底稿第 7 列）
    deduct_rules: Mapped[str] = mapped_column(Text, default="") # 扣分规则（底稿第 8 列）
    common_deductions: Mapped[str] = mapped_column(Text, default="") # 常见扣分情形（黄金数据）
    required_materials: Mapped[str] = mapped_column(Text, default="") # JSON 数组：要求的材料类型
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

class Regulation(Base):
    """法规库条目（v3 §3.1）。
    支持上传原始文件（PDF/Word/TXT），并按文档类型与地区分类：
    - 上位法 / 评价办法 / 编报指南 / 地方法规 / 部门规章 / 其它
    - 国家 / 省 / 市 / 区县 / 部门
    上传后自动解析全文，分块入 Qdrant 向量库供 RAG 检索。
```

```
"""
__tablename__ = "regulations"

id: Mapped[int] = mapped_column(Integer, primary_key=True)
title: Mapped[str] = mapped_column(String(256), index=True)
doc_type: Mapped[str] = mapped_column(String(32), default="其它", index=True)
region: Mapped[str] = mapped_column(String(64), default="国家", index=True)
issuer: Mapped[str] = mapped_column(String(128), default="") # 发文机关
doc_number: Mapped[str] = mapped_column(String(64), default="") # 发文文号
effective_date: Mapped[str] = mapped_column(String(32), default="") # 生效日期 文本
description: Mapped[str] = mapped_column(Text, default="")
tags: Mapped[str] = mapped_column(Text, default="[]") # JSON 数组

# 文件元信息
file_name: Mapped[str] = mapped_column(String(512))
storage_path: Mapped[str] = mapped_column(String(1024))
file_size: Mapped[int] = mapped_column(Integer, default=0)
file_type: Mapped[str] = mapped_column(String(16), default="")

# 解析与索引
parsed_text: Mapped[str] = mapped_column(Text, default="") # 解析后全文（截断）
chunks_count: Mapped[int] = mapped_column(Integer, default=0)
indexed: Mapped[bool] = mapped_column(Boolean, default=False)

uploaded_by: Mapped[Optional[int]] = mapped_column(
    ForeignKey("users.id"), nullable=True
)
created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

class CheckItem(Base):
    """问题清单条目（v3 §3.2）。
    例：Z5-001 真实性 - 材料是否加盖公章、签字齐全、要素完整
    """
    __tablename__ = "check_items"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    item_code: Mapped[str] = mapped_column(String(32), unique=True, index=True)
    # 维度（v3 §1.3）：总体合规性|相关性核查|评分合规性|复核规范性|报告编报合规性
    dimension: Mapped[str] = mapped_column(String(32))
    subcategory: Mapped[str] = mapped_column(String(64), default="") # 真实性/年度一致性/正式性/要素完整性...
    description: Mapped[str] = mapped_column(Text)
    applicable_indicators: Mapped[str] = mapped_column(Text, default="") # JSON: 适用的指标 code 列表（[]=全部）
    risk_level: Mapped[str] = mapped_column(String(8), default="中") # 高|中|低
    common_patterns: Mapped[str] = mapped_column(Text, default="") # JSON 数组
    check_method: Mapped[str] = mapped_column(String(16), default="llm") # rule|llm
    keywords: Mapped[str] = mapped_column(Text, default="") # JSON 数组（rule 用）
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
```

```
# =====
# 被检查单位
# =====
class AuditUnit(Base):
    """被检查单位（v3 §3.7 一个独立角色范畴）。"""
    __tablename__ = "audit_units"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(256), unique=True, index=True)
    code: Mapped[str] = mapped_column(String(64), default="")
    level: Mapped[str] = mapped_column(String(32), default="单位") # 单位 | 部门
    description: Mapped[str] = mapped_column(Text, default="")
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

    tasks: Mapped[List["AuditTask"]] = relationship(back_populates="unit")

# =====
# 核查任务 & 材料
# =====
class AuditTask(Base):
    """一个单位一次核查任务（v3 §3.5）。

    一个任务对应一个被检查单位的内控评价报告核查，包含多个材料和多个指标。
    支持两种核查范围（scope）：
    - all: 对评价指标库内全部已入库指标核查（推荐用于全面体检）
    - selected: 仅对 selected_indicator_ids 列出的指标核查
    """
    __tablename__ = "audit_tasks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    unit_id: Mapped[int] = mapped_column(ForeignKey("audit_units.id"), index=True)
    name: Mapped[str] = mapped_column(String(256))
    eval_year: Mapped[int] = mapped_column(Integer, default=2025)
    # 核查范围
    scope: Mapped[str] = mapped_column(String(16), default="all")
    selected_indicator_ids: Mapped[str] = mapped_column(Text, default="[]") # JSON 数组
    # AI 初核状态: pending → running → ai_done → reviewing(人工复核中) → finalized(已定稿) → archived
    status: Mapped[str] = mapped_column(String(32), default="pending")
    summary: Mapped[str] = mapped_column(Text, default="")
    # 任务级统计（JSON 序列化）
    stats: Mapped[str] = mapped_column(Text, default="")
    # 实时进度（核查时回写）
    progress_current: Mapped[int] = mapped_column(Integer, default=0)
    progress_total: Mapped[int] = mapped_column(Integer, default=0)
    progress_text: Mapped[str] = mapped_column(String(256), default="")
    # 快速模式: 跳过 LLM 思考过程，约 3-4 倍提速
    fast_mode: Mapped[bool] = mapped_column(Boolean, default=False)
```

```

    created_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    completed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    unit: Mapped["AuditUnit"] = relationship(back_populates="tasks")
    materials: Mapped[List["Material"]] = relationship(
        back_populates="task", cascade="all, delete-orphan"
    )
    findings: Mapped[List["Finding"]] = relationship(
        back_populates="task", cascade="all, delete-orphan"
    )
    worksheet: Mapped[Optional["Worksheet"]] = relationship(
        back_populates="task",
        cascade="all, delete-orphan",
        uselist=False,
    )

class Material(Base):
    """被检查单位上传的佐证材料（v3 §3.3）。
    每份材料**必须**绑定一个评价指标（v3 §3.3 明确要求）。
    """
    __tablename__ = "materials"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    task_id: Mapped[int] = mapped_column(ForeignKey("audit_tasks.id"), index=True)
    indicator_id: Mapped[Optional[int]] = mapped_column(
        ForeignKey("indicators.id"), nullable=True, index=True
    )
    file_name: Mapped[str] = mapped_column(String(512))
    storage_path: Mapped[str] = mapped_column(String(1024))
    file_type: Mapped[str] = mapped_column(String(16), default="")
    is_scanned: Mapped[bool] = mapped_column(Boolean, default=False)
    # v3 §3.3 key_elements（公章/签字/日期/文号），JSON 序列化
    key_elements: Mapped[str] = mapped_column(Text, default="")
    parsed_text: Mapped[str] = mapped_column(Text, default="") # 解析后全文（截断）
    # 跨任务查重：MD5(content) 用于精确比对，前 500 字 hash 用于相似比对
    content_hash: Mapped[str] = mapped_column(String(64), default="", index=True)
    content_fingerprint: Mapped[str] = mapped_column(String(64), default="", index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

    task: Mapped["AuditTask"] = relationship(back_populates="materials")
    indicator: Mapped[Optional["Indicator"]] = relationship()

# =====
# 核查发现 & 整改闭环
# =====
class Finding(Base):

```

```
"""AI 核查发现 (v3 §3.4) 。

finding 是 v3 的核心输出对象，每条都对应一个具体的合规疑点，
支持人工复核标注（确认/忽略/调整）和整改闭环。
"""
__tablename__ = "findings"

id: Mapped[int] = mapped_column(Integer, primary_key=True)
task_id: Mapped[int] = mapped_column(ForeignKey("audit_tasks.id"), index=True)
material_id: Mapped[Optional[int]] = mapped_column(
    ForeignKey("materials.id"), nullable=True, index=True
)
indicator_id: Mapped[Optional[int]] = mapped_column(
    ForeignKey("indicators.id"), nullable=True, index=True
)
check_item_id: Mapped[Optional[int]] = mapped_column(
    ForeignKey("check_items.id"), nullable=True
)

# 维度 (v3 §1.3)：真实性|相关性|完整性|合规性|评分合规|复核规范|报告编报
finding_type: Mapped[str] = mapped_column(String(32), default="")
severity: Mapped[str] = mapped_column(String(8), default="中") # 高|中|低
description: Mapped[str] = mapped_column(Text)
evidence_location: Mapped[str] = mapped_column(String(512), default="")
legal_basis: Mapped[str] = mapped_column(Text, default="")
suggestion: Mapped[str] = mapped_column(Text, default="")
source: Mapped[str] = mapped_column(String(16), default="rule") # rule | llm

# 人工复核标注 (v3 §3.5)
review_status: Mapped[str] = mapped_column(String(16), default="pending")
# pending(未复核) | confirmed(确认) | ignored(忽略) | adjusted(调整)
reviewer_id: Mapped[Optional[int]] = mapped_column(
    ForeignKey("users.id"), nullable=True
)
review_note: Mapped[str] = mapped_column(Text, default="")
reviewed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

# 整改闭环 (v3 §3.7)
rectification_status: Mapped[str] = mapped_column(String(16), default="open")
# open(未整改) | submitted(已提交整改) | resolved(已销号)
rectification_note: Mapped[str] = mapped_column(Text, default="")
rectified_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
updated_at: Mapped[datetime] = mapped_column(
    DateTime, server_default=func.now(), onupdate=func.now()
)

task: Mapped["AuditTask"] = relationship(back_populates="findings")
material: Mapped[Optional["Material"]] = relationship()
```

```

        indicator: Mapped[Optional["Indicator"]] = relationship()

# =====
# 工作底稿 (v3 阶段二: AI 阅卷产物)
# =====
class Worksheet(Base):
    """工作底稿头 (与任务 1:1) 。

    阅卷流程: AI 跑完 → 自动建底稿草案 → (V2) 人工复核改单元格 → 定稿 → 报告。
    每条指标对应一行 WorksheetRow。本表存底稿级元数据: 被检查单位名称、组
    织机构代码、核查/复核签名、定稿状态。
    """
    __tablename__ = "worksheets"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    task_id: Mapped[int] = mapped_column(
        ForeignKey("audit_tasks.id"), unique=True, index=True
    )
    unit_name: Mapped[str] = mapped_column(String(256), default="")
    unit_code: Mapped[str] = mapped_column(String(64), default="")
    auditor_name: Mapped[str] = mapped_column(String(64), default="")
    reviewer_name: Mapped[str] = mapped_column(String(64), default="")
    # draft(AI 草稿) | reviewing(复核中) | finalized(已定稿)
    status: Mapped[str] = mapped_column(String(16), default="draft")
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), onupdate=func.now()
    )

    task: Mapped["AuditTask"] = relationship(back_populates="worksheet")
    rows: Mapped[List["WorksheetRow"]] = relationship(
        back_populates="worksheet",
        cascade="all, delete-orphan",
        order_by="WorksheetRow.serial",
    )

class WorksheetRow(Base):
    """工作底稿行 (每条指标在一份底稿里一行, AI 打分粒度) 。"""
    __tablename__ = "worksheet_rows"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    worksheet_id: Mapped[int] = mapped_column(
        ForeignKey("worksheets.id"), index=True
    )
    indicator_id: Mapped[int] = mapped_column(
        ForeignKey("indicators.id"), index=True
    )
    serial: Mapped[int] = mapped_column(Integer, default=0) # 1..55 显示序号

```

```

original_score: Mapped[float] = mapped_column(default=0.0)          # 核查前得分（自评分）
audited_score: Mapped[float] = mapped_column(default=0.0)         # 核查后得分（AI 阅卷）
audit_finding_text: Mapped[str] = mapped_column(Text, default="") # 核查情况说明（兼容旧版）
adjustment_note: Mapped[str] = mapped_column(Text, default="")    # 调整得分说明（V2 新版底稿模板）
# 7 对 14 项复选框 JSON: {real,fake,relevant,irrelevant,effective,ineffective,
# complete,incomplete,compliant,non_compliant,duplicate,unique,match_high,match_low}
material_flags: Mapped[str] = mapped_column(Text, default="{}")
# 反向链接: 本行据以判断的 Finding id 列表 (JSON 数组), 便于追溯
linked_finding_ids: Mapped[str] = mapped_column(Text, default="[]")
created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
updated_at: Mapped[datetime] = mapped_column(
    DateTime, server_default=func.now(), onupdate=func.now()
)

worksheet: Mapped["Worksheet"] = relationship(back_populates="rows")
indicator: Mapped["Indicator"] = relationship()
"""API 请求/响应模型 (v3 内控评价智能审核系统)。"""
from __future__ import annotations

from datetime import datetime
from typing import Any, List, Optional

from pydantic import BaseModel, Field

# =====
# 认证 & 用户
# =====
class LoginRequest(BaseModel):
    username: str
    password: str

class UserOut(BaseModel):
    id: int
    username: str
    role: str
    full_name: str = ""
    unit_id: Optional[int] = None
    is_active: bool = True

    class Config:
        from_attributes = True

class LoginResponse(BaseModel):
    token: str
    user: UserOut
    role_label: str

```

```
class CreateUserRequest(BaseModel):
    username: str
    password: str
    role: str
    full_name: str = ""
    unit_id: Optional[int] = None

class PasswordChangeRequest(BaseModel):
    new_password: str
    old_password: Optional[str] = None # 非管理员改自己时必须填

class SetActiveRequest(BaseModel):
    active: bool

class AuditLogOut(BaseModel):
    id: int
    username: str
    action: str
    target_type: str
    target_id: Optional[int] = None
    detail: str
    created_at: Optional[datetime] = None

    class Config:
        from_attributes = True

# =====
# 系统设置 (LLM API Key 等)
# =====
class LLMSettings(BaseModel):
    provider: str = "stub" # stub | deepseek | claude
    model: str = ""
    base_url: str = ""
    thinking_mode: str = "non_think" # non_think | think_high | think_max
    has_api_key: bool = False # 后端只回是否已配置，不回明文 key

class LLMSettingsUpdate(BaseModel):
    provider: Optional[str] = None
    model: Optional[str] = None
    base_url: Optional[str] = None
    api_key: Optional[str] = None # None=不变, ""=清空, 其他=覆盖
    thinking_mode: Optional[str] = None
```

```
# =====
# 评价指标库 & 问题清单库
# =====
class IndicatorIn(BaseModel):
    indicator_code: str
    level: str = "单位"
    category: str = ""
    subcategory: str = ""
    name: str
    description: str = ""
    max_score: float = 0.0
    audit_points: str = ""
    deduct_rules: str = ""
    common_deductions: str = ""
    required_materials: List[str] = Field(default_factory=list)

class IndicatorOut(BaseModel):
    id: int
    indicator_code: str
    level: str
    category: str
    subcategory: str
    name: str
    description: str
    max_score: float
    audit_points: str
    deduct_rules: str
    common_deductions: str
    required_materials: str
    created_at: Optional[datetime] = None

    class Config:
        from_attributes = True

# =====
# 工作底稿 (AI 阅卷产物)
# =====
class WorksheetRowOut(BaseModel):
    id: int
    indicator_id: int
    serial: int
    original_score: float
    audited_score: float
    audit_finding_text: str
    adjustment_note: str = ""
    material_flags: str
    linked_finding_ids: str
```

```
class Config:
    from_attributes = True

class WorksheetOut(BaseModel):
    id: int
    task_id: int
    unit_name: str
    unit_code: str
    auditor_name: str
    reviewer_name: str
    status: str
    rows: List[WorksheetRowOut] = Field(default_factory=list)
    created_at: Optional[datetime] = None
    updated_at: Optional[datetime] = None

    class Config:
        from_attributes = True

class RegulationOut(BaseModel):
    id: int
    title: str
    doc_type: str
    region: str
    issuer: str
    doc_number: str
    effective_date: str
    description: str
    tags: str
    file_name: str
    file_size: int
    file_type: str
    chunks_count: int
    indexed: bool
    uploaded_by: Optional[int] = None
    created_at: Optional[datetime] = None

    class Config:
        from_attributes = True

class RegulationListResponse(BaseModel):
    regulations: List[RegulationOut]
    total: int
    doc_types: List[str]
    regions: List[str]

class CheckItemIn(BaseModel):
```

```
    item_code: str
    dimension: str
    subcategory: str = ""
    description: str
    applicable_indicators: List[str] = Field(default_factory=list)
    risk_level: str = "中"
    common_patterns: List[str] = Field(default_factory=list)
    check_method: str = "llm" # rule | llm
    keywords: List[str] = Field(default_factory=list)

class CheckItemOut(BaseModel):
    id: int
    item_code: str
    dimension: str
    subcategory: str
    description: str
    applicable_indicators: str
    risk_level: str
    common_patterns: str
    check_method: str
    keywords: str
    is_active: bool

    class Config:
        from_attributes = True

# =====
# 被检查单位
# =====
class AuditUnitIn(BaseModel):
    name: str
    code: str = ""
    level: str = "单位"
    description: str = ""

class AuditUnitOut(BaseModel):
    id: int
    name: str
    code: str
    level: str
    description: str
    created_at: Optional[datetime] = None

    class Config:
        from_attributes = True
```

```
# =====
# 核查任务 & 材料 & 核查发现
# =====
class AuditTaskCreate(BaseModel):
    unit_id: int
    name: str
    eval_year: int = 2025
    scope: str = "all" # "all" | "selected"
    selected_indicator_ids: List[int] = Field(default_factory=list)
    fast_mode: bool = False

class AuditTaskOut(BaseModel):
    id: int
    unit_id: int
    name: str
    eval_year: int
    scope: str = "all"
    selected_indicator_ids: str = "[]"
    status: str
    summary: str
    stats: str
    progress_current: int = 0
    progress_total: int = 0
    progress_text: str = ""
    fast_mode: bool = False
    created_at: Optional[datetime] = None
    completed_at: Optional[datetime] = None

    class Config:
        from_attributes = True

class MaterialOut(BaseModel):
    id: int
    task_id: int
    indicator_id: Optional[int] = None
    file_name: str
    file_type: str
    is_scanned: bool
    key_elements: str

    class Config:
        from_attributes = True

class FindingOut(BaseModel):
    id: int
    task_id: int
    material_id: Optional[int] = None
```

```
indicator_id: Optional[int] = None
check_item_id: Optional[int] = None
finding_type: str
severity: str
description: str
evidence_location: str
legal_basis: str
suggestion: str
source: str
review_status: str
reviewer_id: Optional[int] = None
review_note: str
rectification_status: str
rectification_note: str
created_at: Optional[datetime] = None

class Config:
    from_attributes = True

class TaskDetailOut(BaseModel):
    task: AuditTaskOut
    unit: AuditUnitOut
    materials: List[MaterialOut] = []
    findings: List[FindingOut] = []

class FindingReviewRequest(BaseModel):
    status: str # confirmed | ignored | adjusted
    note: str = ""

class FindingRectifyRequest(BaseModel):
    note: str

class FindingRectifyConfirmRequest(BaseModel):
    note: str = ""
"""核查任务 / 材料 / Finding API (v3) 。"""
from __future__ import annotations

from typing import List, Optional

from fastapi import APIRouter, Depends, File, Form, HTTPException, Query, UploadFile
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from sqlalchemy.orm import Session

from app.api.schemas import (
    AuditTaskCreate,
```

```
AuditTaskOut,
AuditUnitIn,
AuditUnitOut,
FindingOut,
FindingRectifyConfirmRequest,
FindingRectifyRequest,
FindingReviewRequest,
MaterialOut,
TaskDetailOut,
WorksheetOut,
)
from app.core.auth import get_current_user, log_action, require_auditor
from app.core.permissions import is_admin, is_auditor_or_above, is_unit
from app.models import AuditTask, AuditUnit, Finding, Indicator, Material, User, get_db
from app.parsers.dispatcher import UnsupportedFormatError
from app.services import audit_service
from app.services.report_service import build_report_docx
from app.services.worksheet_export import build_worksheet_xlsx
from app.services.worksheet_service import build_worksheet_draft, get_worksheet
from app.tasks import run_audit_task

units_router = APIRouter(prefix="/api/units", tags=["audit:units"])
tasks_router = APIRouter(prefix="/api/tasks", tags=["audit:tasks"])
findings_router = APIRouter(prefix="/api/findings", tags=["audit:findings"])
materials_router = APIRouter(prefix="/api/materials", tags=["audit:materials"])

# =====
# 单位
# =====
@units_router.get("", response_model=List[AuditUnitOut])
def list_units(db: Session = Depends(get_db),
               user: User = Depends(get_current_user)):
    q = db.query(AuditUnit).order_by(AuditUnit.id.desc())
    if is_unit(user.role) and user.unit_id:
        q = q.filter(AuditUnit.id == user.unit_id)
    return q.all()

@units_router.post("", response_model=AuditUnitOut)
def create_unit(req: AuditUnitIn,
               db: Session = Depends(get_db),
               user: User = Depends(require_auditor)):
    return audit_service.create_unit(
        db, name=req.name, code=req.code,
        level=req.level, description=req.description, user=user,
    )

@units_router.delete("/{unit_id}")
```



```
def delete_unit(unit_id: int,
                db: Session = Depends(get_db),
                user: User = Depends(require_auditor)):
    """删除被检查单位（含任务引用校验）。"""
    unit = db.get(AuditUnit, unit_id)
    if not unit:
        raise HTTPException(404, "单位不存在")
    n_tasks = db.query(AuditTask).filter(AuditTask.unit_id == unit_id).count()
    if n_tasks > 0:
        raise HTTPException(
            400,
            f"该单位下还有 {n_tasks} 个核查任务，请先删除任务再删除单位",
        )
    # 把绑定的 unit 用户解绑
    db.query(User).filter(User.unit_id == unit_id).update({User.unit_id: None})
    name = unit.name
    db.delete(unit)
    log_action(db, user, "unit.delete",
              target_type="unit", target_id=unit_id,
              detail=f"删除单位「{name}」")
    db.commit()
    return {"status": "ok"}

# =====
# 任务
# =====
def _user_can_see_task(user: User, task: AuditTask) -> bool:
    if is_auditor_or_above(user.role):
        return True
    if is_unit(user.role) and user.unit_id == task.unit_id:
        return True
    return False

@tasks_router.get("", response_model=List[AuditTaskOut])
def list_tasks(db: Session = Depends(get_db),
               user: User = Depends(get_current_user)):
    q = db.query(AuditTask).order_by(AuditTask.id.desc())
    if is_unit(user.role) and user.unit_id:
        q = q.filter(AuditTask.unit_id == user.unit_id)
    return q.all()

@tasks_router.post("", response_model=AuditTaskOut)
def create_task(req: AuditTaskCreate,
                db: Session = Depends(get_db),
                user: User = Depends(require_auditor)):
    return audit_service.create_task(
        db, unit_id=req.unit_id, name=req.name,
```

```
        eval_year=req.eval_year,
        scope=req.scope,
        selected_indicator_ids=req.selected_indicator_ids,
        fast_mode=req.fast_mode,
        user=user,
    )

@tasks_router.get("/{task_id}", response_model=TaskDetailOut)
def get_task_detail(task_id: int,
                    db: Session = Depends(get_db),
                    user: User = Depends(get_current_user)):
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权查看此任务")
    unit = db.get(AuditUnit, task.unit_id)

    # 老任务的 stats 里没 scoring → 临时回填一次（不写回 DB，纯展示）
    if task.status in ("ai_done", "reviewing", "finalized", "archived"):
        import json as _json
        try:
            stats = _json.loads(task.stats or "{}")
        except Exception:
            stats = {}
        if "scoring" not in stats:
            try:
                from app.services.scoring_service import compute_task_scoring
                stats["scoring"] = compute_task_scoring(db, task)
                task.stats = _json.dumps(stats, ensure_ascii=False)
                db.commit()
            except Exception as exc:
                print(f"[scoring backfill] {exc}")
                db.rollback()

    return TaskDetailOut(
        task=AuditTaskOut.model_validate(task),
        unit=AuditUnitOut.model_validate(unit),
        materials=[MaterialOut.model_validate(m) for m in task.materials],
        findings=[FindingOut.model_validate(f) for f in task.findings],
    )

@tasks_router.post("/{task_id}/materials", response_model=MaterialOut)
async def upload_material(
    task_id: int,
    file: UploadFile = File(...),
    indicator_id: Optional[int] = Form(None),
    db: Session = Depends(get_db),
```

```
        user: User = Depends(get_current_user),
    ):
        task = db.get(AuditTask, task_id)
        if not task:
            raise HTTPException(404, "任务不存在")
        if not _user_can_see_task(user, task):
            raise HTTPException(403, "无权上传到此任务")
        try:
            content = await file.read()
            material = audit_service.upload_material(
                db, task, file_name=file.filename or "untitled",
                content=content, indicator_id=indicator_id, user=user,
            )
        except UnsupportedFormatError as exc:
            raise HTTPException(400, str(exc))
        return material

class MaterialBindRequest(BaseModel):
    indicator_id: Optional[int] = None # null=解绑

@tasks_router.patch("/{task_id}/materials/{material_id}", response_model=MaterialOut)
def bind_material_indicator(task_id: int, material_id: int,
                           req: MaterialBindRequest,
                           db: Session = Depends(get_db),
                           user: User = Depends(require_auditor)):
    """改/解绑某份材料的指标。"""
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权操作此任务")
    material = db.get(Material, material_id)
    if not material or material.task_id != task_id:
        raise HTTPException(404, "材料不存在或不属于本任务")
    if req.indicator_id is not None:
        ind = db.get(Indicator, req.indicator_id)
        if not ind:
            raise HTTPException(400, f"指标 {req.indicator_id} 不存在")
    old_iid = material.indicator_id
    material.indicator_id = req.indicator_id
    log_action(db, user, "material.bind",
               target_type="material", target_id=material.id,
               detail=f"任务 #{task_id} 材料 {material.file_name} 绑定 "
                    f"{old_iid} → {req.indicator_id}")
    db.commit(); db.refresh(material)
    return material
```

```
@tasks_router.post("/{task_id}/materials/auto-bind")
def auto_bind_materials(task_id: int,
                        db: Session = Depends(get_db),
                        user: User = Depends(require_auditor)):
    """对未绑定指标的材料按文件名关键词批量自动绑定。"""
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权操作此任务")
    return audit_service.auto_bind_materials(db, task, user)

@tasks_router.post("/{task_id}/run", response_model=AuditTaskOut)
def run_task(task_id: int,
             force: bool = Query(False, description="已完成的任务必须设 true 才允许重跑"),
             db: Session = Depends(get_db),
             user: User = Depends(require_auditor)):
    """触发 AI 核查（异步入队，eager 模式立即返回 done）。
    防呆：
    - status=running: 拒绝（避免并行重复跑）
    - status in ai_done/reviewing/finalized/archived: 要求 force=true（避免误点重跑）
    """
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not task.materials:
        raise HTTPException(400, "任务下尚无材料，请先上传")
    if task.status == "running":
        raise HTTPException(400, "任务正在核查中，请等待完成后再操作")
    if task.status in ("ai_done", "reviewing", "finalized", "archived") and not force:
        raise HTTPException(
            400,
            "任务已完成核查。重新核查会清空已有疑点与工作底稿，请确认后带 force=true 参数",
        )
    log_action(db, user, "task.run",
              target_type="task", target_id=task.id,
              detail=f"触发 AI 核查 ({len(task.materials)} 份材料"
                    f"'{'', 强制重跑' if force else ''}) ")
    db.commit()
    run_audit_task.delay(task.id)
    db.refresh(task)
    return task

@tasks_router.post("/{task_id}/finalize", response_model=AuditTaskOut)
def finalize_task(task_id: int,
                 db: Session = Depends(get_db),
                 user: User = Depends(require_auditor)):
```

```
        return audit_service.finalize_task(db, task_id, user)

@tasks_router.delete("/{task_id}")
def delete_task(task_id: int,
                db: Session = Depends(get_db),
                user: User = Depends(require_auditor)):
    """删除任务（级联清理材料 + finding + 物理文件）。仅审查员及以上可删。"""
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权删除此任务")
    audit_service.delete_task(db, task_id, user)
    return {"status": "ok"}

@tasks_router.get("/{task_id}/report")
def download_task_report(task_id: int,
                        db: Session = Depends(get_db),
                        user: User = Depends(get_current_user)):
    """生成 Word 核查报告（v3 §3.6 5 章节）。"""
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权下载此任务报告")
    try:
        data = build_report_docx(db, task)
    except Exception as exc:
        raise HTTPException(500, f"报告生成失败: {exc}")
    safe_name = f"内控评价核查报告_{task.eval_year}_{task.id}.docx"
    # RFC 5987 编码中文文件名
    from urllib.parse import quote
    filename_quoted = quote(safe_name)
    return StreamingResponse(
        iter([data]),
        media_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        headers={
            "Content-Disposition":
                f"attachment; filename=report_{task.id}.docx; "
                f"filename*=UTF-8''{filename_quoted}"
        },
    )

# =====
# 材料审核聚合视图（4 类数据）
# =====
@tasks_router.get("/{task_id}/material-review")
```

```
def get_material_review(task_id: int,
                        db: Session = Depends(get_db),
                        user: User = Depends(get_current_user)):
    """聚合: 重复检测 / 内容审核 / 匹配情况 / 操作时间线。"""
    from app.services import material_review_service
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权查看")
    return material_review_service.review_overview(db, task)

class MergeDuplicatesRequest(BaseModel):
    content_hash: str
    keep_material_id: int

@tasks_router.post("/{task_id}/materials/merge-duplicates")
def merge_duplicate_materials(task_id: int,
                              req: MergeDuplicatesRequest,
                              db: Session = Depends(get_db),
                              user: User = Depends(require_auditor)):
    """同任务内 content_hash 相同的多份材料, 保留 keep_material_id, 删除其余。

    被删材料关联的 Finding 不删, 只 material_id 设 NULL (保留可追溯)。
    """
    from app.services import material_review_service
    task = db.get(AuditTask, task_id)
    if not task:
        raise HTTPException(404, "任务不存在")
    if not _user_can_see_task(user, task):
        raise HTTPException(403, "无权操作此任务")
    res = material_review_service.merge_duplicate_group(
        db, task, req.content_hash, req.keep_material_id,
        user_id=user.id if user else None,
    )
    if res.get("error"):
        raise HTTPException(400, res["error"])
    log_action(db, user, "material.merge",
              target_type="task", target_id=task.id,
              detail=f"合并重复材料: 保留 #{res['kept']}, 删除 {res['removed']} 份")
    db.commit()
    return res

# =====
# 工作底稿 (AI 阅卷 → 底稿 → 报告)
# =====
@tasks_router.get("/{task_id}/worksheet", response_model=WorksheetOut)
```

```
        const m2 = cd.match(/filename="?([^\"]+)"?/);
        if (m2) fname = m2[1];
    }
    const a = document.createElement("a");
    a.href = url;
    a.download = fname;
    document.body.appendChild(a);
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 1000);
    toast("已下载, 请用本地 Office 打开", "success");
}
} catch (e) {
    toast("打开失败: " + (e.message || e), "error");
}
};

window.downloadRegulation = async function(id) {
    const tok = getToken();
    if (!tok) { toast("请先登录", "error"); return; }
    try {
        toast("正在下载...");
        const r = await fetch(`${API}/regulations/${id}/download`, {
            headers: { Authorization: "Bearer " + tok },
        });
        if (!r.ok) {
            const text = await r.text();
            throw new Error(text || `HTTP ${r.status}`);
        }
        const blob = await r.blob();
        // 从 Content-Disposition 解析文件名 (含 RFC 5987 中文)
        const cd = r.headers.get("Content-Disposition") || "";
        let fname = `regulation_${id}`;
        const m = cd.match(/filename\*=UTF-8'('([^']*))/);
        if (m) fname = decodeURIComponent(m[1]);
        else {
            const m2 = cd.match(/filename="?([^\"]+)"?/);
            if (m2) fname = m2[1];
        }
        const url = URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url;
        a.download = fname;
        document.body.appendChild(a);
        a.click();
        a.remove();
        setTimeout(() => URL.revokeObjectURL(url), 1000);
        toast("✓ 下载完成", "success");
    } catch (e) {
        toast("下载失败: " + (e.message || e), "error");
    }
};
```

```
    }  
  };  
  
window.previewRegulation = async function(id) {  
  toast("加载法规内容...");  
  try {  
    const data = await api(`/regulations/${id}/content`);  
    const r = data.regulation;  
    document.getElementById("rp-title").textContent = r.title;  
    document.getElementById("rp-doctype").innerHTML = docTypeBadge(r.doc_type);  
    document.getElementById("rp-region").innerHTML = `  
    const metaParts = [];  
    if (r.issuer) metaParts.push(esc(r.issuer));  
    if (r.doc_number) metaParts.push(`    if (r.effective_date) metaParts.push(`生效 ${esc(r.effective_date)}`);  
    metaParts.push(`文件 ${esc(r.file_name)} (${(r.file_size / 1024).toFixed(1)} KB)`);  
    metaParts.push(`    document.getElementById("rp-meta").innerHTML = metaParts.join(" · ");  
  
    document.getElementById("rp-text").textContent = data.text;  
    const trunc = document.getElementById("rp-truncated");  
    if (data.truncated) {  
      trunc.textContent = ` (已截取前 ${data.text.length} / 全文 ${data.text_total_chars} 字符) `;  
      trunc.classList.remove("hidden");  
    } else {  
      trunc.classList.add("hidden");  
    }  
  
    document.getElementById("rp-chunks-count").textContent = data.chunks.length;  
    document.getElementById("rp-chunks-list").innerHTML = data.chunks.length  
      ? data.chunks.map((c, i) => `  
        <div style="border-bottom:1px solid var(--divider);padding:12px 8px">  
          <div class="flex items-center gap-2 mb-2">  
            <span class="code-id">#${pad(i + 1)}</span>  
            <span class="badge badge-blue">${esc(c.citation)}</span>  
            ${c.article ? `<span class="text-xs text-faint">${esc(c.article)}</span>` : ''}  
          </div>  
          <div style="font-size:13px;line-height:1.6;color:var(--text-secondary);white-space:pre-wrap">${esc(c.tex  
t)}</div>  
        </div>`).join("")  
      : `  
    const rpDl = document.getElementById("rp-download");  
    rpDl.removeAttribute("href");  
    rpDl.style.cursor = "pointer";  
    rpDl.onclick = (ev) => { ev.preventDefault(); downloadRegulation(id); };  
  
    // 默认显示全文 tab  
    document.querySelectorAll("[data-rptab]").forEach(b => {  
      b.classList.toggle("active", b.dataset.rptab === "text");  
    });  
  }  
};
```



```
});
document.getElementById("rp-tab-text").classList.remove("hidden");
document.getElementById("rp-tab-chunks").classList.add("hidden");

document.getElementById("reg-preview-modal").classList.remove("hidden");
} catch (e) {
  toast("预览失败: " + e.message, "error");
}
};

// 预览模式的 tab 切换
document.addEventListener("click", ev => {
  if (!ev.target.matches("[data-rptab]")) return;
  const t = ev.target.dataset.rptab;
  document.querySelectorAll("[data-rptab]").forEach(b => b.classList.toggle("active", b === ev.target));
  document.getElementById("rp-tab-text").classList.toggle("hidden", t !== "text");
  document.getElementById("rp-tab-chunks").classList.toggle("hidden", t !== "chunks");
});

window.deleteRegulation = async function(id, title) {
  if (!confirm(`确定删除法规《${title}》? \n该法规的原始文件将被删除（向量库内的索引块保留）。`)) return;
  try {
    await api(`/regulations/${id}`, { method: "DELETE" });
    toast("✓ 已删除", "success");
    loadRegulations();
  } catch (e) { toast(e.message, "error"); }
};

// 搜索 / 筛选 实时触发
let _regSearchTimer;
function bindRegFilters() {
  const searchInput = document.getElementById("reg-search");
  if (searchInput && !searchInput._bound) {
    searchInput._bound = true;
    searchInput.addEventListener("input", () => {
      clearTimeout(_regSearchTimer);
      _regSearchTimer = setTimeout(loadRegulations, 280);
    });
    document.getElementById("reg-filter-type").addEventListener("change", loadRegulations);
    document.getElementById("reg-filter-region").addEventListener("change", loadRegulations);
  }
}
bindRegFilters();

// 上传弹窗
// =====
// 文件夹批量上传（功能 4）
// =====
const FOLDER_STATE = {
  files: [],
```

```
    strategy: "ai",
    cancelled: false,
  };
  const FOLDER_MAX_FILES = 200;
  const FOLDER_MAX_SIZE_MB = 50;
  const FOLDER_CONCURRENCY = 3;
  const SUPPORTED_FOLDER_EXTS = [".pdf", ".docx", ".xlsx", ".txt", ".md"];
  const PATH_FOLDER_DOC_TYPES = ["上位法", "评价办法", "编报指南", "地方法规",
    "部门规章", "高频问题"];

  document.getElementById("open-folder-upload").addEventListener("click", () => {
    document.getElementById("folder-picker").click();
  });

  document.getElementById("folder-picker").addEventListener("change", ev => {
    const all = Array.from(ev.target.files || []);
    ev.target.value = ""; // 允许再次选择同一文件夹
    if (!all.length) return;

    // 按扩展名过滤
    const valid = all.filter(f => {
      const name = (f.name || "").toLowerCase();
      return SUPPORTED_FOLDER_EXTS.some(ext => name.endsWith(ext));
    });

    if (!valid.length) {
      toast("文件夹内未找到任何支持的文件（PDF/Word/Excel/TXT/MD）", "error");
      return;
    }
    if (valid.length > FOLDER_MAX_FILES) {
      toast(`文件数 ${valid.length} 超过 ${FOLDER_MAX_FILES} 上限，请分批上传`, "error");
      return;
    }
    const tooBig = valid.filter(f => f.size > FOLDER_MAX_SIZE_MB * 1024 * 1024);
    if (tooBig.length) {
      toast(`${tooBig.length} 份文件超过 ${FOLDER_MAX_SIZE_MB}MB，请检查`, "error");
      return;
    }

    FOLDER_STATE.files = valid;
    FOLDER_STATE.cancelled = false;
    openFolderConfigModal();
  });

  function openFolderConfigModal() {
    document.getElementById("fc-count").textContent = FOLDER_STATE.files.length;
    document.getElementById("folder-config-modal").classList.remove("hidden");
    document.getElementById("fc-error").classList.add("hidden");
    document.getElementById("fc-strategy").value = "ai";
    updateFcStrategyUI("ai");
  }
}
```

```
}

document.getElementById("fc-strategy").addEventListener("change", ev => {
  updateFcStrategyUI(ev.target.value);
});

function updateFcStrategyUI(s) {
  document.getElementById("fc-strategy-ai").classList.toggle("hidden", s !== "ai");
  document.getElementById("fc-strategy-uniform").classList.toggle("hidden", s !== "uniform");
  document.getElementById("fc-strategy-path").classList.toggle("hidden", s !== "path");
  if (s === "path") renderPathPreview();
}

function fileRelativePath(f) {
  // 浏览器把目录相对路径放在 webkitRelativePath (如 "上位法/财办63号.pdf")
  return f.webkitRelativePath || f.name;
}

function renderPathPreview() {
  // 按一级目录分组预览
  const groups = {};
  FOLDER_STATE.files.forEach(f => {
    const rel = fileRelativePath(f);
    const parts = rel.split("/");
    // parts[0] 是用户选的根目录名; parts[1] 是真正的一级子目录 (如 "上位法")
    const subDir = parts.length >= 3 ? parts[1] : "(根目录)";
    groups[subDir] = (groups[subDir] || 0) + 1;
  });
  const box = document.getElementById("fc-path-preview");
  box.innerHTML = Object.entries(groups)
    .map(([dir, n]) => {
      const matched = PATH_FOLDER_DOC_TYPES.includes(dir);
      const cls = matched ? "color:var(--green)" : "color:var(--orange)";
      const label = matched ? `→ ${dir}` : `→ 其它 (未匹配)`;
      return `<div style="${cls}">· <b>${esc(dir)}</b> (${n} 份) <span style="opacity:0.7">${label}</span></div>`;
    })
    .join("");
}

document.getElementById("fc-start").addEventListener("click", async () => {
  const strategy = document.getElementById("fc-strategy").value;
  FOLDER_STATE.strategy = strategy;

  // AI 策略需要 API Key
  if (strategy === "ai") {
    try {
      const cfg = await api("/settings/llm");
      if (!cfg.has_api_key) {
        document.getElementById("fc-error").textContent =
          "AI 策略需要先在「后台管理 → 大语言模型」配置 API Key, 请改选「统一应用」或「按路径」";
        document.getElementById("fc-error").classList.remove("hidden");
      }
    }
  }
});
```

```
        return;
    }
} catch (e) {
    document.getElementById("fc-error").textContent = "无法读取 LLM 配置: " + e.message;
    document.getElementById("fc-error").classList.remove("hidden");
    return;
}
}

document.getElementById("folder-config-modal").classList.add("hidden");
await runFolderUpload();
});

// =====
// 主上传循环
// =====
async function runFolderUpload() {
    const files = FOLDER_STATE.files;
    const strategy = FOLDER_STATE.strategy;
    FOLDER_STATE.cancelled = false;

    // 准备进度模态
    const tbody = document.getElementById("fp-tbody");
    document.getElementById("fp-close").classList.add("hidden");
    document.getElementById("fp-cancel").classList.remove("hidden");
    document.getElementById("fp-summary").textContent =
        `共 ${files.length} 份文件, 使用「${strategyLabel(strategy)}」策略`;
    document.getElementById("fp-bar").style.width = "0%";

    tbody.innerHTML = files.map((f, idx) => `
        <tr id="fp-row-${idx}">
            <td><span id="fp-icon-${idx}" class="text-muted">📁</span></td>
            <td style="word-break:break-all">${esc(fileRelativePath(f))}</td>
            <td id="fp-doctype-${idx}" class="text-faint">—</td>
            <td id="fp-region-${idx}" class="text-faint">—</td>
            <td id="fp-chunks-${idx}" class="text-faint">—</td>
            <td id="fp-msg-${idx}" class="text-sm text-muted">待上传</td>
        </tr>`).join("");

    document.getElementById("folder-progress-modal").classList.remove("hidden");

    // 并发控制
    let cursor = 0;
    let done = 0;
    let succeeded = 0;
    let failed = 0;

    const updateBar = () => {
        document.getElementById("fp-bar").style.width = `${(done / files.length) * 100}%`;
        document.getElementById("fp-summary").textContent =
```

```
`进度 ${done}/${files.length} · 成功 ${succeeded} · 失败 ${failed}`;
};

async function worker() {
  while (true) {
    if (FOLDER_STATE.cancelled) break;
    const idx = cursor++;
    if (idx >= files.length) break;
    const file = files[idx];
    await processOne(file, idx, strategy)
      .then(() => { succeeded++; })
      .catch(() => { failed++; });
    done++;
    updateBar();
  }
}

const workers = Array.from({ length: FOLDER_CONCURRENCY }, () => worker());
await Promise.all(workers);

// 完成
document.getElementById("fp-cancel").classList.add("hidden");
document.getElementById("fp-close").classList.remove("hidden");
if (FOLDER_STATE.cancelled) {
  document.getElementById("fp-summary").textContent =
    `已取消 · 完成 ${done}/${files.length} · 成功 ${succeeded} · 失败 ${failed}`;
  toast(`已取消上传 (已完成 ${done})`, "info");
} else {
  toast(`✓ 完成: 成功 ${succeeded}, 失败 ${failed}`, succeeded > 0 ? "success" : "error");
}
// 刷新法规库列表
loadRegulations();
}

function strategyLabel(s) {
  return { ai: "AI 智能识别", uniform: "统一应用", path: "按路径" }[s] || s;
}

document.getElementById("fp-cancel").addEventListener("click", () => {
  FOLDER_STATE.cancelled = true;
  document.getElementById("fp-cancel").disabled = true;
});

// =====
// 单文件处理: 决策分类 → POST 上传
// =====
async function processOne(file, idx, strategy) {
  setRowState(idx, "wait", "正在处理...");
  let meta;
  try {
```

```
    meta = await decideClassification(file, strategy);
  } catch (e) {
    setRowState(idx, "fail", "分类失败: " + e.message);
    return Promise.reject(e);
  }

  setRowDoc(idx, meta);
  setRowState(idx, "wait", "上传中...");

  try {
    const fd = new FormData();
    fd.append("file", file);
    fd.append("title", meta.title);
    fd.append("doc_type", meta.doc_type);
    fd.append("region", meta.region);
    fd.append("issuer", meta.issuer || "");
    fd.append("doc_number", meta.doc_number || "");
    fd.append("effective_date", meta.effective_date || "");
    fd.append("description", `批量上传 · 策略: ${strategyLabel(strategy)}`);
    fd.append("tags", "[]");

    const tok = getToken();
    const r = await fetch(`${API}/regulations`, {
      method: "POST",
      headers: { "Authorization": "Bearer " + tok },
      body: fd,
    });
    if (!r.ok) {
      const text = await r.text();
      let msg = text;
      try { msg = JSON.parse(text).detail || text; } catch {}
      throw new Error(msg || `HTTP ${r.status}`);
    }
    const reg = await r.json();
    setRowChunks(idx, reg.chunks_count);
    setRowState(idx, "ok", `✓ 已上传`);
    return reg;
  } catch (e) {
    setRowState(idx, "fail", "x " + e.message);
    throw e;
  }
}

async function decideClassification(file, strategy) {
  if (strategy === "uniform") {
    return {
      title: stripExt(file.name),
      doc_type: document.getElementById("fc-default-doc-type").value,
      region: document.getElementById("fc-default-region").value,
    };
  }
}
```

```
}

if (strategy === "path") {
  const parts = fileRelativePath(file).split("/");
  const subDir = parts.length >= 3 ? parts[1] : "";
  const doc_type = PATH_FOLDER_DOC_TYPES.includes(subDir) ? subDir : "其它";
  // 简单按文件夹名推断 region
  let region = "国家";
  if (doc_type === "地方法规") region = "省";
  if (doc_type === "部门规章") region = "部门";
  return {
    title: stripExt(file.name),
    doc_type,
    region,
  };
}

// ai 策略: 调 /classify 端点
const fd = new FormData();
fd.append("file", file);
const tok = getToken();
const r = await fetch(`${API}/regulations/classify`, {
  method: "POST",
  headers: { "Authorization": "Bearer " + tok },
  body: fd,
});
if (!r.ok) {
  const text = await r.text();
  let msg = text;
  try { msg = JSON.parse(text).detail || text; } catch {}
  throw new Error(msg);
}
return await r.json();
}

function stripExt(name) {
  const i = name.lastIndexOf(".");
  return i > 0 ? name.slice(0, i) : name;
}

function setRowState(idx, state, msg) {
  const icon = document.getElementById(`fp-icon-${idx}`);
  const m = document.getElementById(`fp-msg-${idx}`);
  if (icon) {
    const iconMap = {
      wait: '<span style="color:var(--orange)">⌛</span>',
      ok: '<span style="color:var(--green)">✓</span>',
      fail: '<span style="color:var(--red)">✗</span>',
    };
    icon.innerHTML = iconMap[state] || "o";
  }
}
```

```
}
if (m) {
  m.textContent = msg;
  m.className = "text-sm " + (state === "fail" ? "" : "text-muted");
  if (state === "fail") m.style.color = "var(--red)";
}
}

function setRowDoc(idx, meta) {
  const dt = document.getElementById(`fp-doctype-${idx}`);
  const rg = document.getElementById(`fp-region-${idx}`);
  if (dt) {
    dt.innerHTML = docTypeBadge(meta.doc_type);
  }
  if (rg) {
    rg.innerHTML = `${esc(meta.region)}`;
  }
}

function setRowChunks(idx, n) {
  const c = document.getElementById(`fp-chunks-${idx}`);
  if (c) c.innerHTML = `${n}`;
}

// =====
// 原有单文件上传按钮
// =====
document.getElementById("open-reg-upload").addEventListener("click", () => {
  document.getElementById("reg-upload-modal").classList.remove("hidden");
  document.getElementById("reg-upload-error").classList.add("hidden");
  document.getElementById("reg-upload-status").innerHTML = "";
  document.getElementById("reg-upload-form").reset();
});

document.getElementById("reg-upload-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const fd = new FormData(ev.target);
  const status = document.getElementById("reg-upload-status");
  const errBox = document.getElementById("reg-upload-error");
  errBox.classList.add("hidden");

  // FormData 已包含 file + 所有 input; 确保 tags 是合法 JSON
  fd.set("tags", "[]");

  status.innerHTML = `

第 40 页 / 共 60 页


```



```
        body: fd,
      });
      if (!r.ok) {
        const err = await r.json().catch(() => ({ detail: "上传失败" }));
        throw new Error(err.detail || "上传失败");
      }
      const reg = await r.json();
      status.innerHTML = `<div class="callout callout-success">✓ 上传成功 · ${reg.chunks_count} 个条款块已入向量库</div>`;
    };
    setTimeout(() => {
      document.getElementById("reg-upload-modal").classList.add("hidden");
      loadRegulations();
    }, 800);
  } catch (e) {
    errBox.textContent = e.message;
    errBox.classList.remove("hidden");
    status.innerHTML = "";
  }
});

// =====
// 知识库 - 评价指标 / 问题清单 (增强: 搜索 + 删除)
// =====
let _indCache = [];

async function loadIndicators() {
  const inds = await api("/indicators");
  State.indicators = inds;
  _indCache = inds;

  // 装填业务分类下拉
  const fcat = document.getElementById("ind-filter-category");
  if (fcat.options.length <= 1) {
    const cats = [...new Set(inds.map(i => i.category).filter(Boolean))].sort();
    fcat.innerHTML = `<option value="">全部业务分类</option>` +
      cats.map(c => `<option value="${esc(c)}">${esc(c)}</option>`).join("");
  }
  renderIndicators();
}

function renderIndicators() {
  const search = document.getElementById("ind-search").value.trim().toLowerCase();
  const level = document.getElementById("ind-filter-level").value;
  const category = document.getElementById("ind-filter-category").value;

  let filtered = _indCache;
  if (level) filtered = filtered.filter(i => i.level === level);
  if (category) filtered = filtered.filter(i => i.category === category);
  if (search) {
    filtered = filtered.filter(i =>
      (i.indicator_code || "").toLowerCase().includes(search) ||

```

```

        (i.name || "").toLowerCase().includes(search) ||
        (i.category || "").toLowerCase().includes(search) ||
        (i.subcategory || "").toLowerCase().includes(search)
    );
}

document.getElementById("ind-count").textContent = `共 ${filtered.length} / ${_indCache.length} 条`;
const tbody = document.getElementById("indicators-tbody");
if (!filtered.length) {
    tbody.innerHTML = `<tr><td colspan="7" class="empty-state">
        <div class="empty-state-glyph">⦿</div>
        ${_indCache.length === 0 ? '暂无评价指标，使用右上角「批量导入 JSON」开始' : '当前筛选下无结果'}
    </td></tr>`;
    return;
}
const isAdmin = State.user && State.user.role === "super_admin";
tbody.innerHTML = filtered.map(i => {
    let mats = []; try { mats = JSON.parse(i.required_materials || "[]"); } catch {}
    return `<tr>
        <td><span class="code-id">${esc(i.indicator_code)}</span></td>
        <td><span class="badge badge-gray">${esc(i.level)}</span></td>
        <td>${esc(i.category)}</td>
        <td style="font-weight:500">${esc(i.name)}</td>
        <td class="table-mono">${i.max_score}</td>
        <td class="text-sm text-muted">${esc(mats.join("\n"))} || '-'</td>
        <td class="text-right" style="white-space:nowrap">
            <button class="btn btn-ghost btn-sm" onclick="viewIndicator(${i.id})" title="查看详情">${icon("view")}</but
ton>
            ${isAdmin
                ? `<button class="btn btn-danger-ghost btn-sm" onclick="deleteIndicator(${i.id}, '${esc(i.indicator_cod
e)}')` title="删除">${icon("delete")}</button>`
                : ''}
            </td>
        </tr>`;
    }).join("");
}

// 查看评价指标详情
window.viewIndicator = function(id) {
    const ind = _indCache.find(i => i.id === id);
    if (!ind) return;
    let mats = []; try { mats = JSON.parse(ind.required_materials || "[]"); } catch {}

    const rows = [
        ["指标编号", `<span class="code-id">${esc(ind.indicator_code)}</span>`],
        ["层级", `<span class="badge badge-gray">${esc(ind.level)}</span>`],
        ["业务分类", esc(ind.category) || "-"],
        ["子类", esc(ind.subcategory) || "-"],
        ["满分", `<span class="table-mono">${ind.max_score} 分</span>`],
        ["指标描述", esc(ind.description) || "-"],
        ["扣分细则", `<div class="detail-quote" style="margin:0">${esc(ind.deduct_rules)} || "-"</div>`],
        ["常见扣分情形", `<div class="detail-quote" style="margin:0">${esc(ind.common_deductions)} || "-"</div>`],
    ]

```

```
["必需材料", mats.length ? mats.map(m => `
```

```
document.getElementById("ind-filter-category").addEventListener("change", renderIndicators);
}
}
bindIndFilters();

let _ciCache = [];

async function loadCheckItems() {
  const items = await api("/check-items");
  _ciCache = items;
  renderCheckItems();
}

function renderCheckItems() {
  const search = document.getElementById("ci-search").value.trim().toLowerCase();
  const dim = document.getElementById("ci-filter-dim").value;
  const method = document.getElementById("ci-filter-method").value;

  let filtered = _ciCache;
  if (dim) filtered = filtered.filter(x => x.dimension === dim);
  if (method) filtered = filtered.filter(x => x.check_method === method);
  if (search) {
    filtered = filtered.filter(x =>
      (x.item_code || "").toLowerCase().includes(search) ||
      (x.description || "").toLowerCase().includes(search) ||
      (x.subcategory || "").toLowerCase().includes(search)
    );
  }

  document.getElementById("ci-count").textContent = `共 ${filtered.length} / ${_ciCache.length} 条`;
  const tbody = document.getElementById("items-tbody");
  if (!filtered.length) {
    tbody.innerHTML = `|  |  |  |  |  |  |  |
| --- | --- | --- | --- | --- | --- | --- |
| <div class="empty-state-glyph">⦿</div>       ${_ciCache.length === 0 ? '暂无条目, 请批量导入' : '当前筛选下无结果'}     </td></tr>`;     return;   }   const isAdmin = State.user && State.user.role === "super_admin";   tbody.innerHTML = filtered.map(it => `|     <td><span class="code-id">${esc(it.item_code)}</span></td>     <td><span class="badge badge-blue">${esc(it.dimension)}</span></td>     <td>${esc(it.subcategory)}</td>     <td class="text-sm">${esc(it.description)}</td>     <td><span class="tag">${esc(checkMethodLabel(it.check_method))}</span></td>     <td><span class="chip-risk chip-risk-${it.risk_level}">${it.risk_level}</span></td>     <td class="text-right" style="white-space: nowrap">       <button class="btn btn-ghost btn-sm" onclick="viewCheckItem(${it.id})" title="查看详情">${icon("view")}</butt on>       ${isAdmin         ? `<button class="btn btn-danger-ghost btn-sm" onclick="deleteCheckItem(${it.id}, '${esc(it.item_code)}')`         : ''}       title="删除">${icon("delete")}</button>`   )} |  | | | | | | | |

```

```
        : ''}
      </td>
    </tr>`).join("");
  }

// 查看问题清单详情
window.viewCheckItem = function(id) {
  const it = _ciCache.find(x => x.id === id);
  if (!it) return;

  let apps = []; try { apps = JSON.parse(it.applicable_indicators || "[]"); } catch {}
  let pats = []; try { pats = JSON.parse(it.common_patterns || "[]"); } catch {}
  let kws = []; try { kws = JSON.parse(it.keywords || "[]"); } catch {}

  const rows = [
    ["条目编号", `
```

```
// 通用: 导出 JSON 为浏览器下载
function downloadJsonBlob(filename, data) {
  const json = JSON.stringify(data, null, 2);
  const blob = new Blob([json], { type: "application/json;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  URL.revokeObjectURL(url);
}

// 批量导出当前筛选列表
document.getElementById("export-indicators").addEventListener("click", () => {
  // 用当前筛选后的列表
  const search = (document.getElementById("ind-search").value || "").trim().toLowerCase();
  const level = document.getElementById("ind-filter-level").value;
  const category = document.getElementById("ind-filter-category").value;
  let list = _indCache;
  if (level) list = list.filter(i => i.level === level);
  if (category) list = list.filter(i => i.category === category);
  if (search) {
    list = list.filter(i =>
      (i.indicator_code || "").toLowerCase().includes(search) ||
      (i.name || "").toLowerCase().includes(search) ||
      (i.category || "").toLowerCase().includes(search)
    );
  }
  if (!list.length) { toast("当前筛选下没有可导出条目", "error"); return; }
  const data = list.map(i => {
    let mats = []; try { mats = JSON.parse(i.required_materials || "[]"); } catch {}
    return {
      indicator_code: i.indicator_code,
      level: i.level,
      category: i.category,
      subcategory: i.subcategory,
      name: i.name,
      description: i.description,
      max_score: i.max_score,
      deduct_rules: i.deduct_rules,
      common_deductions: i.common_deductions,
      required_materials: mats,
    };
  });
  const ts = new Date().toISOString().slice(0, 10);
  downloadJsonBlob(`indicators_${ts}_${data.length}items.json`, data);
  toast(`✓ 已导出 ${data.length} 条评价指标`, "success");
});
```

```
});

document.getElementById("export-check-items").addEventListener("click", () => {
  const search = (document.getElementById("ci-search").value || "").trim().toLowerCase();
  const dim = document.getElementById("ci-filter-dim").value;
  const method = document.getElementById("ci-filter-method").value;
  let list = _ciCache;
  if (dim) list = list.filter(x => x.dimension === dim);
  if (method) list = list.filter(x => x.check_method === method);
  if (search) {
    list = list.filter(x =>
      (x.item_code || "").toLowerCase().includes(search) ||
      (x.description || "").toLowerCase().includes(search)
    );
  }
  if (!list.length) { toast("当前筛选下没有可导出条目", "error"); return; }
  const data = list.map(it => {
    let apps = []; try { apps = JSON.parse(it.applicable_indicators || "[]"); } catch {}
    let pats = []; try { pats = JSON.parse(it.common_patterns || "[]"); } catch {}
    let kws = []; try { kws = JSON.parse(it.keywords || "[]"); } catch {}
    return {
      item_code: it.item_code,
      dimension: it.dimension,
      subcategory: it.subcategory,
      description: it.description,
      applicable_indicators: apps,
      risk_level: it.risk_level,
      common_patterns: pats,
      check_method: it.check_method,
      keywords: kws,
    };
  });
  const ts = new Date().toISOString().slice(0, 10);
  downloadJsonBlob(`check_items_${ts}_${data.length}items.json`, data);
  toast(`✓ 已导出 ${data.length} 条问题清单`, "success");
});

window.deleteCheckItem = async function(id, code) {
  if (!confirm(`确定删除问题清单「${code}」? (软删, 可在管理端恢复)`)) return;
  try {
    await api(`/check-items/${id}`, { method: "DELETE" });
    toast("✓ 已删除", "success");
    loadCheckItems();
  } catch (e) { toast(e.message, "error"); }
};

let _ciSearchTimer;
function bindCiFilters() {
  const s = document.getElementById("ci-search");
  if (s && !s._bound) {
```

```
s._bound = true;
s.addEventListener("input", () => {
  clearTimeout(_ciSearchTimer);
  _ciSearchTimer = setTimeout(renderCheckItems, 200);
});
document.getElementById("ci-filter-dim").addEventListener("change", renderCheckItems);
document.getElementById("ci-filter-method").addEventListener("change", renderCheckItems);
}
}
bindCiFilters();

async function uploadJson(endpoint, file) {
  const fd = new FormData(); fd.append("file", file);
  const tok = getToken();
  const r = await fetch(API + endpoint, {
    method: "POST", headers: { "Authorization": "Bearer " + tok }, body: fd,
  });
  if (!r.ok) throw new Error(await r.text());
  return r.json();
}

// =====
// 批量导入 (PDF / Word / Excel / JSON) + 预览确认流程
// =====

let _importState = {
  kind: null,      // "indicators" | "check-items"
  file: null,
  preview: [],
  total: 0,
};

const IMPORT_COLUMNS = {
  "indicators": [
    { key: "indicator_code", label: "编号", w: 70 },
    { key: "category", label: "指标分类", w: 130 },
    { key: "name", label: "指标名称", w: 160 },
    { key: "max_score", label: "标准分", w: 60 },
    { key: "audit_points", label: "核查要点", w: 260, fmt: truncate160 },
    { key: "deduct_rules", label: "扣分规则", w: 260, fmt: truncate160 },
  ],
  "check-items": [
    { key: "item_code", label: "编号", w: 90 },
    { key: "dimension", label: "维度", w: 120 },
    { key: "subcategory", label: "子类", w: 100 },
    { key: "description", label: "描述" },
    { key: "check_method", label: "方法", w: 80, fmt: checkMethodLabel },
    { key: "risk_level", label: "风险", w: 60 },
  ],
};
```



```
function tryJsonList(v) {
  try {
    const arr = typeof v === "string" ? JSON.parse(v) : v;
    return Array.isArray(arr) ? arr.join("\n") : String(v);
  } catch { return String(v); }
}

function truncate160(v) {
  const s = String(v ?? "");
  return s.length > 160 ? s.slice(0, 160) + "..." : s;
}

// 问题清单 check_method 字段中文化
const CHECK_METHOD_LABEL = {
  rule: "规则匹配",
  llm: "AI 分析",
};
function checkMethodLabel(m) {
  return CHECK_METHOD_LABEL[m] || (m || "-");
}

async function importDryRun(kind, file) {
  const endpoint = kind === "indicators"
    ? "/indicators/import-from-file?dry_run=true"
    : "/check-items/import-from-file?dry_run=true";
  const fd = new FormData(); fd.append("file", file);
  const tok = getToken();
  const r = await fetch(API + endpoint, {
    method: "POST",
    headers: { "Authorization": "Bearer " + tok },
    body: fd,
  });
  if (!r.ok) {
    const text = await r.text();
    let msg = text;
    try { msg = JSON.parse(text).detail || text; } catch {}
    throw new Error(msg);
  }
  return r.json();
}

async function importConfirm(kind, file) {
  const endpoint = kind === "indicators"
    ? "/indicators/import-from-file?dry_run=false"
    : "/check-items/import-from-file?dry_run=false";
  const fd = new FormData(); fd.append("file", file);
  const tok = getToken();
  const r = await fetch(API + endpoint, {
    method: "POST",
  });
}
```

```

    headers: { "Authorization": "Bearer " + tok },
    body: fd,
  });
  if (!r.ok) {
    const text = await r.text();
    let msg = text;
    try { msg = JSON.parse(text).detail || text; } catch {}
    throw new Error(msg);
  }
  return r.json();
}

function openImportPreview(kind, file, resp) {
  _importState = { kind, file, preview: resp.preview || [], total: resp.total || resp.preview.length };

  const labels = {
    "indicators": { title: "评价指标 - 导入预览", page: "评价指标" },
    "check-items": { title: "问题清单 - 导入预览", page: "问题清单" },
  };
  document.getElementById("ipm-title").textContent = labels[kind].title;
  document.getElementById("ipm-sub").textContent =
    `文件: ${file.name} · 抽取条目: ${_importState.total} (预览前 ${_importState.preview.length} 条)`;

  // note 样式: Excel 表头识别 = 绿色 (快/准), LLM 抽取 = 蓝色, 正则 = 黄色
  const note = String(resp.note || "");
  let kind_color = "info";
  if (note.includes("Excel 表头自动识别")) kind_color = "success";
  else if (note.includes("LLM 抽取")) kind_color = "info";
  else if (note.includes("正则启发式")) kind_color = "warn";
  document.getElementById("ipm-note").innerHTML = note
    ? `<div class="callout callout-${kind_color}">${esc(note)}</div>`
    : "";

  const cols = IMPORT_COLUMNS[kind];
  const thead = document.getElementById("ipm-thead");
  thead.innerHTML = `<tr>${cols.map(c =>
    `<th${c.w ? ` style="width:${c.w}px"` : ''}>${esc(c.label)}</th>`).join("")}</tr>`;

  const tbody = document.getElementById("ipm-tbody");
  if (!_importState.preview.length) {
    tbody.innerHTML = `<tr><td colspan="${cols.length}" class="empty-state">
      <div class="empty-state-glyph">⦿</div>未抽到条目, 请检查文件内容或配置 LLM API Key 后重试
    </td></tr>`;
    document.getElementById("ipm-confirm").disabled = true;
  } else {
    tbody.innerHTML = _importState.preview.map(row => `<tr>${cols.map(c => {
      let v = row[c.key];
      if (c.fmt) v = c.fmt(v);
      const txt = v == null ? '-' : String(v);
      return `<td class="text-sm" style="white-space:pre-wrap;line-height:1.5;vertical-align:top">${esc(txt)}</td>`
    })}`);
  }
};

```

```
    }).join("}")</tr>`).join("");
    document.getElementById("ipm-confirm").disabled = false;
  }
  document.getElementById("ipm-status").innerHTML = "";
  document.getElementById("import-preview-modal").classList.remove("hidden");
}

async function handleImportSelect(kind, ev) {
  const file = ev.target.files[0];
  if (!file) return;
  ev.target.value = ""; // 允许重选同文件
  toast(`正在解析 ${file.name}...`);
  try {
    const resp = await importDryRun(kind, file);
    openImportPreview(kind, file, resp);
  } catch (e) {
    toast(`✖ 解析失败: ${e.message}`, "error");
  }
}

document.getElementById("import-indicators").addEventListener("change",
  ev => handleImportSelect("indicators", ev));
document.getElementById("import-items").addEventListener("change",
  ev => handleImportSelect("check-items", ev));

document.getElementById("ipm-confirm").addEventListener("click", async () => {
  const { kind, file } = _importState;
  if (!kind || !file) return;
  const btn = document.getElementById("ipm-confirm");
  const status = document.getElementById("ipm-status");
  btn.disabled = true;
  btn._t = btn.textContent;
  btn.textContent = "导入中...";
  status.innerHTML = `<div class="callout callout-info">正在写入数据库...</div>`;
  try {
    const result = await importConfirm(kind, file);
    const errCount = (result.errors || []).length;
    status.innerHTML = `<div class="callout callout-success">
      ✓ 完成: 新建 ${result.created}, 跳过 ${result.skipped}${errCount ? ` , 错误 ${errCount}` : ''}
    </div>`;
    toast(`✓ 已导入 ${result.created} 条${result.skipped ? ` (跳过 ${result.skipped})` : ''} : ''`, "success");
    setTimeout(() => {
      document.getElementById("import-preview-modal").classList.add("hidden");
      if (kind === "indicators") loadIndicators();
      else loadCheckItems();
    }, 1000);
  } catch (e) {
    status.innerHTML = `<div class="callout callout-error">✖ ${esc(e.message)}</div>`;
    toast(`✖ 导入失败: ${e.message}`, "error");
  } finally {
    btn.disabled = false;
    btn.textContent = btn._t;
    status.innerHTML = "";
  }
});
```

```
    btn.disabled = false;
    btn.textContent = btn._t || "确认导入";
  }
});

// =====
// 后台管理控制台
// =====
function setConsoleTab(tab) {
  State.consoleTab = tab;
  document.querySelectorAll(".console-nav-item").forEach(b =>
    b.classList.toggle("active", b.dataset.cnav === tab));
  document.querySelectorAll(".console-panel").forEach(p => p.classList.add("hidden"));
  document.getElementById("console-" + tab).classList.remove("hidden");
  switch (tab) {
    case "llm": loadLLMConfig(); break;
    case "system": loadSystemInfo(); break;
    case "users": loadUsers(); break;
    case "units": loadUnitsConsole(); break;
    case "audit": loadAuditLogs(); break;
  }
}

document.querySelectorAll(".console-nav-item").forEach(b => {
  b.addEventListener("click", () => setConsoleTab(b.dataset.cnav));
});

// LLM 配置
async function loadLLMConfig() {
  try {
    const cfg = await api("/settings/llm");
    const form = document.getElementById("llm-form");
    form.provider.value = cfg.provider;
    form.model.value = cfg.model;
    form.base_url.value = cfg.base_url;
    form.thinking_mode.value = cfg.thinking_mode;
    form.api_key.value = "";
    document.getElementById("llm-key-hint").textContent = cfg.has_api_key
      ? "✓ 已配置 API Key · 留空表示不修改"
      : "尚未配置 · 请填写 DeepSeek API Key (sk-...) ";
    document.getElementById("llm-status").textContent = "";
  } catch (e) { console.error(e); }
}

document.getElementById("llm-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const form = ev.target;
  const submitBtn = form.querySelector('button[type="submit"]');
  const fd = new FormData(form);
  const status = document.getElementById("llm-status");
```

```
const payload = {
  provider: fd.get("provider"), model: fd.get("model"),
  base_url: fd.get("base_url"), thinking_mode: fd.get("thinking_mode"),
};
const apiKey = fd.get("api_key");
if (apiKey !== "") payload.api_key = apiKey.trim();

// 立即反馈 (toast + callout + 按钮 loading)
status.innerHTML = `<div class="callout callout-info">正在保存配置...</div>`;
if (submitBtn) {
  submitBtn._origText = submitBtn._origText || submitBtn.textContent;
  submitBtn.disabled = true;
  submitBtn.textContent = "保存中...";
}
toast("正在保存 LLM 配置...");

try {
  const result = await api("/settings/llm", {
    method: "PUT", headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  const provider = result.provider || payload.provider;
  const hasKey = result.has_api_key ? "已配置 API Key" : "未配置 API Key";
  status.innerHTML = `<div class="callout callout-success">✓ 已保存 · ${esc(provider)} · ${esc(hasKey)}</div>`;
  toast(`✓ 已保存: ${provider} (${hasKey})`, "success");
  await loadLLMConfig();
} catch (e) {
  status.innerHTML = `<div class="callout callout-error">✗ 保存失败: ${esc(e.message)}</div>`;
  toast(`✗ 保存失败: ${e.message}`, "error");
} finally {
  if (submitBtn) {
    submitBtn.disabled = false;
    submitBtn.textContent = submitBtn._origText || "保存配置";
  }
}
});

document.getElementById("llm-test").addEventListener("click", async () => {
  const btn = document.getElementById("llm-test");
  const status = document.getElementById("llm-status");
  btn._origText = btn._origText || btn.textContent;
  btn.disabled = true;
  btn.textContent = "测试中...";
  status.innerHTML = `<div class="callout callout-info">正在向 LLM 发送测试请求...</div>`;
  toast("正在测试 LLM 连接...");
  try {
    const r = await api("/settings/llm/test", { method: "POST" });
    if (r.success) {
      const preview = (r.preview || "").slice(0, 60);
      status.innerHTML = `<div class="callout callout-success">✓ ${esc(r.client)} 连接成功${preview ? ' · 响应: ' + esc(preview) : ''}</div>`;
    }
  }
});
```

```
    toast(`✓ ${r.client} 连接成功`, "success");
  } else {
    status.innerHTML = `

x ${esc(r.client)}: ${esc(r.error)}</div>`;
    toast(`x 测试失败: ${r.error}`, "error");
  }
} catch (e) {
  status.innerHTML = `

x ${esc(e.message)}</div>`;
  toast(`x ${e.message}`, "error");
} finally {
  btn.disabled = false;
  btn.textContent = btn._origText || "测试连接";
}
});

// 系统信息
async function loadSystemInfo() {
  try {
    const h = await api("/health");
    document.getElementById("system-info").innerHTML = `
      <div class="kv-row">
        <div class="kv-key">系统名称</div>
        <div class="kv-value">${esc(h.app)}</div>
      </div>
      <div class="kv-row">
        <div class="kv-key">服务状态</div>
        <div class="kv-value"><span class="badge badge-green">● 正常运行</span></div>
      </div>
      <div class="kv-row">
        <div class="kv-key">LLM 默认服务商</div>
        <div class="kv-value"><span class="tag">${esc(h.llm_default_provider)}</span></div>
      </div>
      <div class="kv-row">
        <div class="kv-key">Embedding 引擎</div>
        <div class="kv-value"><span class="tag">${esc(h.embedder)}</span></div>
      </div>
      <div class="kv-row">
        <div class="kv-key">向量库</div>
        <div class="kv-value"><span class="tag">${esc(h.vector_store)}</span></div>
      </div>
      <div class="kv-row">
        <div class="kv-key">前端版本</div>
        <div class="kv-value text-muted">v3.0 · 苹果系统级 UI</div>
      </div>`;
  } catch (e) { console.error(e); }
}

// 用户
async function loadUsers() {
  const users = await api("/users");
  const meId = State.user && State.user.id;


```

```

document.getElementById("users-tbody").innerHTML = users.map(u => {
  const isSelf = u.id === meId;
  const isDeleted = u.username.startsWith("deleted_");
  const activeBadge = u.is_active
    ? '<span class="badge badge-green">启用</span>'
    : (isDeleted ? '<span class="badge badge-red">已删除</span>' : '<span class="badge badge-gray">停用</span>');
  const actions = [];
  actions.push('<button class="btn btn-ghost btn-sm" onclick="openChangePwd(${u.id}, '${esc(u.username)}')'" title="
e="改密码">${icon("key")}</button>');
  if (!isSelf && !isDeleted) {
    if (u.is_active) {
      actions.push('<button class="btn btn-ghost btn-sm" onclick="toggleUserActive(${u.id}, false)" title="停
用">${icon("pause")}</button>');
    } else {
      actions.push('<button class="btn btn-ghost btn-sm" onclick="toggleUserActive(${u.id}, true)" title="启用">
${icon("play")}</button>');
    }
    actions.push('<button class="btn btn-danger-ghost btn-sm" onclick="deleteUser(${u.id}, '${esc(u.usernam
e)}')'" title="软删">${icon("delete")}</button>');
  }
  return `<tr>
    <td><span class="code-id">#${pad(u.id)}</span></td>
    <td style="font-weight:500">${esc(u.username)}${isSelf ? ' <span class="text-xs text-faint">(你)</span>' :
''}</td>
    <td>${roleBadge(u.role)}</td>
    <td>${esc(u.full_name || "-")}</td>
    <td>${activeBadge}</td>
    <td class="text-right" style="white-space:nowrap">${actions.join(" ")</td>
  </tr>`;
}).join("") || `<tr><td colspan="6" class="empty-state">暂无用户</td></tr>`;
}

// 改密码 / 启停 / 删除 - 全局处理器
window.openChangePwd = function(userId, username) {
  const isSelf = State.user && State.user.id === userId;
  const isAdmin = State.user && State.user.role === "super_admin";
  const needOld = !isAdmin && isSelf;
  const modal = document.getElementById("pwd-modal");
  document.getElementById("pwd-title").textContent =
    isSelf ? "修改我的密码" : `重置 ${username} 的密码`;
  document.getElementById("pwd-target-id").value = userId;
  document.getElementById("pwd-form").reset();
  document.getElementById("pwd-old-row").style.display = needOld ? "" : "none";
  document.getElementById("pwd-status").innerHTML = "";
  document.getElementById("pwd-old-input").required = needOld;
  modal.classList.remove("hidden");
};

window.toggleUserActive = async function(userId, active) {
  const verb = active ? "启用" : "停用";
  if (!confirm(`确定${verb}此用户? \n${active ? "" : "停用后该用户当前登录态将立即失效。"}`)) return;
  try {
    await api(`/users/${userId}/activate`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
    });
  }

```

```
        body: JSON.stringify({ active }),
      });
      toast(`✓ 已${verb}`, "success");
      loadUsers();
    } catch (e) { toast(e.message, "error"); }
  };

window.deleteUser = async function(userId, username) {
  if (!confirm(`确定删除用户 ${username}? \n\n  软删（操作日志中保留）\n  该用户立即无法登录\n  用户名会加 deleted_ 前缀\n\n此操作不可恢复。`)) return;
  try {
    await api(`/users/${userId}`, { method: "DELETE" });
    toast("✓ 用户已删除", "success");
    loadUsers();
  } catch (e) { toast(e.message, "error"); }
};

// 改密码表单提交
document.getElementById("pwd-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const fd = new FormData(ev.target);
  const userId = parseInt(document.getElementById("pwd-target-id").value);
  const newPwd = fd.get("new_password");
  const confirmPwd = fd.get("new_password_confirm");
  const oldPwd = fd.get("old_password");
  const status = document.getElementById("pwd-status");

  if (newPwd !== confirmPwd) {
    status.innerHTML = `<div class="callout callout-warn">两次输入的新密码不一致</div>`;
    return;
  }
  if (newPwd.length < 6) {
    status.innerHTML = `<div class="callout callout-warn">密码至少 6 位</div>`;
    return;
  }

  try {
    const payload = { new_password: newPwd };
    if (oldPwd) payload.old_password = oldPwd;
    await api(`/users/${userId}/password`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    });
    document.getElementById("pwd-modal").classList.add("hidden");
    const isSelf = State.user && State.user.id === userId;
    toast(`✓ 密码已修改${isSelf ? ", 请重新登录" : ""}`, "success");
    if (isSelf) {
      // 改自己密码 → token 已被吊销 → 退出
      setToken(""); State.user = null;
      setTimeout(() => showLogin("密码已修改, 请用新密码登录"), 500);
    }
  }
});
```



```
    } else {
      // 改别人的密码（管理员场景）→ 刷新用户列表
      if (typeof loadUsers === "function") loadUsers();
    }
  } catch (e) {
    status.innerHTML = `<div class="callout callout-error">× ${esc(e.message)}</div>`;
  }
});

function roleBadge(r) {
  const map = {
    super_admin: ['badge badge-brand', '超级管理员'],
    auditor: ['badge badge-blue', '审查员'],
    unit: ['badge badge-orange', '被检查单位'],
    readonly: ['badge badge-gray', '只读'],
  };
  const [cls, label] = map[r] || ['badge badge-gray', r];
  return `<span class="${cls}">${label}</span>`;
}

document.getElementById("user-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const fd = new FormData(ev.target);
  const status = document.getElementById("user-form-status");
  try {
    await api("/users", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        username: fd.get("username"), password: fd.get("password"),
        role: fd.get("role"), full_name: fd.get("full_name") || "",
      }),
    });
    status.innerHTML = `<div class="callout callout-success">✓ 用户已创建</div>`;
    ev.target.reset();
    loadUsers();
  } catch (e) {
    status.innerHTML = `<div class="callout callout-error">× ${esc(e.message)}</div>`;
  }
});

// 单位
async function loadUnitsConsole() {
  const units = await api("/units");
  document.getElementById("units-tbody").innerHTML = units.map(u => `
    <tr>
      <td><span class="code-id">#${pad(u.id)}</span></td>
      <td style="font-weight:500">${esc(u.name)}</td>
      <td class="text-muted">${esc(u.code || "-")}</td>
      <td><span class="badge badge-gray">${esc(u.level)}</span></td>
      <td class="text-sm text-muted">${fmtTime(u.created_at)}</td>
    </tr>`
  );
}
```

```
        <td class="text-right">
          <button class="btn btn-danger-ghost btn-sm" onclick="deleteUnit(${u.id}, '${esc(u.name).replace(/'/g,
"\\'")})'" title="删除单位">${icon("delete")}</button>
        </td>
      </tr>`).join("") || `<tr><td colspan="6" class="empty-state">暂无单位</td></tr>`;
    }

window.deleteUnit = async function(unitId, name) {
  if (!confirm(`确定删除单位「${name}」？\n\n如有关联任务必须先删除任务。`)) return;
  try {
    await api(`/units/${unitId}`, { method: "DELETE" });
    toast(`✓ 已删除单位「${name}」`, "success");
    await loadUnitsConsole();
  } catch (e) {
    toast(`✗ ${e.message}`, "error");
  }
};

document.getElementById("unit-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const fd = new FormData(ev.target);
  const status = document.getElementById("unit-form-status");
  try {
    await api("/units", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        name: fd.get("name"), code: fd.get("code"), level: fd.get("level"),
      }),
    });
    status.innerHTML = `<div class="callout callout-success">✓ 单位已创建</div>`;
    ev.target.reset();
    loadUnitsConsole();
  } catch (e) {
    status.innerHTML = `<div class="callout callout-error">✗ ${esc(e.message)}</div>`;
  }
});

// 审计日志
async function loadAuditLogs() {
  const logs = await api("/audit-logs?limit=100");
  document.getElementById("audit-tbody").innerHTML = logs.map(l => `
    <tr>
      <td class="text-sm text-muted">${fmtTime(l.created_at)}</td>
      <td style="font-weight:500">${esc(l.username || "-")}</td>
      <td><span class="tag">${esc(l.action)}</span></td>
      <td class="text-sm">${esc(l.target_type)}${l.target_id ? ' #' + l.target_id : ''}</td>
      <td class="text-sm text-muted">${esc(l.detail || "-")}</td>
    </tr>`).join("") || `<tr><td colspan="5" class="empty-state">暂无日志</td></tr>`;
}
document.getElementById("refresh-audit").addEventListener("click", loadAuditLogs);
```

```
// =====
// 认证 & 启动
// =====
function showLogin(msg) {
  document.getElementById("app").classList.add("hidden");
  document.getElementById("login-shell").classList.remove("hidden");
  const err = document.getElementById("login-error");
  if (msg) { err.textContent = msg; err.classList.remove("hidden"); }
  else { err.classList.add("hidden"); }
}
function hideLogin() {
  document.getElementById("login-shell").classList.add("hidden");
  document.getElementById("app").classList.remove("hidden");
}

function renderUserCard() {
  const box = document.getElementById("user-card");
  if (!State.user) { box.innerHTML = ""; return; }
  const name = State.user.full_name || State.user.username;
  box.innerHTML = `
    <div class="user-card">
      <div class="user-avatar">${initial(name)}</div>
      <div class="user-meta">
        <div class="user-name">${esc(name)}</div>
        <div class="user-role">${esc(State.roleLabel)}</div>
      </div>
      <button class="btn-logout-icon" id="btn-changePwd" title="改密码">${icon("key")}</button>
      <button class="btn-logout-icon" id="btn-logout" title="登出">
        <svg width="14" height="14" viewBox="0 0 16 16" fill="none">
          <path d="M6 3H3v10h3 M11 11l3-3-3 M14 8H6" stroke="currentColor" stroke-width="1.5" stroke-linecap="ro
und" stroke-linejoin="round"/>
        </svg>
      </button>
    </div>`;
  document.getElementById("btn-changePwd").addEventListener("click", () => {
    openChangePwd(State.user.id, State.user.username);
  });
  document.getElementById("btn-logout").addEventListener("click", async () => {
    try { await api("/auth/logout", { method: "POST" }); } catch {}
    setToken(""); State.user = null;
    showLogin();
  });
}

// 按角色显隐侧栏每一条导航项
document.querySelectorAll(".nav-link").forEach(btn => {
  const route = btn.dataset.route;
  const visible = isRouteAllowed(route, State.user);
  btn.style.display = visible ? "" : "none";
});
// 管理分组：里面所有项都不可见时整段隐藏
const adminSection = document.getElementById("nav-admin-section");
```

```
const adminVisible = isRouteAllowed("console", State.user);
adminSection.style.display = adminVisible ? "" : "none";
// 知识库分组: 里面所有项都不可见时整段隐藏
const kbSection = document.querySelector('[data-nav-group="knowledge"]');
if (kbSection) {
  const kbVisible = isRouteAllowed("indicators", State.user) ||
    isRouteAllowed("check-items", State.user);
  kbSection.style.display = kbVisible ? "" : "none";
}
}
}

document.getElementById("login-form").addEventListener("submit", async ev => {
  ev.preventDefault();
  const fd = new FormData(ev.target);
  try {
    const r = await fetch(API + "/auth/login", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username: fd.get("username"), password: fd.get("password") }),
    });
    if (!r.ok) {
      const err = await r.json().catch(() => ({ detail: "登录失败" }));
      throw new Error(err.detail || "登录失败");
    }
    const data = await r.json();
    setToken(data.token);
    State.user = data.user;
    State.roleLabel = data.role_label;
    hideLogin();
    renderUserCard();
    if (!location.hash) location.hash = "#/dashboard";
    handleRoute();
  } catch (e) { showLogin(e.message); }
});

async function bootstrap() {
  if (getToken()) {
    try {
      const data = await api("/auth/me");
      State.user = data.user;
      State.roleLabel = data.role_label;
      hideLogin();
      renderUserCard();
      if (!location.hash) location.hash = "#/dashboard";
      handleRoute();
      return;
    } catch {}
  }
  showLogin();
}
bootstrap();
```